

AA 228 Quiz 0 Cheatsheet

Probability Review & useful identities

$$\bullet P(E \cup F) = P(E) + P(F) - P(E \cap F)$$

PMF: probability mass function (for discrete distributions)

PDF: probability density function (for continuous distributions)

CDF:

$$CDF_X(x) = P(X \leq x) = \int_{-\infty}^x \underbrace{p(x')}_{\text{where } p(x) \text{ is the PDF}} dx'$$

Gaussian Distribution: $X \sim \mathcal{N}(\sigma, \mu^2)$ $\rightarrow$ 2 parameters

Uniform Distribution: $X \sim U(\alpha, \beta)$ $\rightarrow$ 2 parameters

$$f(x) = \begin{cases} 1/\beta - \alpha & \alpha \leq x \leq \beta \\ 0 & \text{otherwise} \end{cases} \quad \begin{aligned} \mu &= \frac{1}{2}(\alpha + \beta) \\ \sigma^2 &= \frac{1}{12}(\beta - \alpha)^2 \end{aligned}$$

Marginal Distribution: $p(x) = \sum_y P(x, y)$ or $p(x) = \int P(x, y) dy$

Conditional Probability $\triangleq P(X|Y) = P(X \cap Y) / P(Y)$

$$\Rightarrow P(X \cap Y) = P(X|Y) P(Y)$$

LEADS TO BAYES THEOREM:

$$P(X|Y) = \frac{P(Y|X) P(X)}{P(Y)}$$

REPRESENTATIONS

Concept: # of independent variables = $2^n - 1$ where n is the # of binary parameters you have.

Law: if X & Y are independent $P(X, Y) = P(X)P(Y)$

"independent" $\triangleq$ the outcome of X does not impact the prob. dist. of Y

Trees as a structure: reduces the # of independent probabilities that need storage

If X has n values & Y has m values, the # of independent values of $P(X|Y)$ = $(n-1) * m$

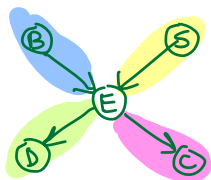
BAYESIAN NETWORKS

- directed, acyclic
- set of CONDITIONAL probability distributions

Computing the probability of a "particular assignment" of $x_{1:n}$ nodes in a Bayesian Network:

$$P(x_{1:n}) = \prod_{i=1}^n P(x_i | Pa(x_i))$$

Practice: What is the Σ # of independent parameters in the following Bayes Net?



$$B: 1$$

$$S: 1$$

$$E: (2-1)(2) + (2-1)(2) = 4$$

$$D: (2-1)(2) = 2$$

$$C: (2-1)(2) = 2$$

$$\Sigma = 10 \text{ variables}$$

where

5 unrelated binary variables have **31**

$$\text{Conditional independence} \triangleq P(X, Y | Z) = P(X | Z) P(Y | Z) \triangleq (X \perp Y | Z)$$

$$\text{property: } (X \perp Y | Z) \text{ iff } P(X | Z) = P(X | Y, Z)$$

- D-separation Rules
- ① D-separated if path contains a chain $(X) \rightarrow (Y) \rightarrow (Z)$ s.t. $Y \in C$
 - ② " " fork $(X) \leftarrow (Y) \rightarrow (Z)$ s.t. $Y \in C$
 - ③ The path contains an inverted fork (v-structure) s.t. $Y \notin C$ & no descendant of Y is in C $(X) \rightarrow (Y) \leftarrow (Z)$

INFERENCE: infer distribution over query variables given evidence variables.

Ex: $P(b' | c', d')$ $\rightarrow$ trying to "infer" b given c & d .

$$\text{Cond. prob: } P(b' | c', d') = \frac{P(b', c', d')}{P(c', d')}$$

$$\text{Marginalization: } P(b', c', d') = \sum_s \sum_e P(b', s, e, c', d') \quad // \text{ sum over hidden variables } s \text{ \& } e$$

FACTORS: factor $\triangleq$ a discrete, multivariate distribution.

"a function from assignments of random variables to real #'s."

Basic factor operations:

$$\textcircled{1} \text{ Factor product: } \phi(X, Y) * \phi(Y, Z) = \phi(X, Y, Z)$$

X	Y	$\phi_1(X, Y)$
0	0	0.3
0	1	0.4
1	0	0.2
1	1	0.1

Y	Z	$\phi_2(Y, Z)$
0	0	0.2
0	1	0.0
1	0	0.3
1	1	0.5

X	Y	Z	$\phi_3(X, Y, Z)$
0	0	0	0.06
0	0	1	0.00
0	1	0	0.12
0	1	1	0.20
1	0	0	0.04
1	0	1	0.00
1	1	0	0.03
1	1	1	0.05

X	Y	Z	$\phi(X, Y, Z)$
0	0	0	0.08
0	0	1	0.31
0	1	0	0.09
0	1	1	0.37
1	0	0	0.01
1	0	1	0.05
1	1	0	0.02
1	1	1	0.07

X	Z	$\phi(X, Z)$
0	0	0.17
0	1	0.68
1	0	0.03
1	1	0.12

$$\textcircled{2} \text{ Factor marginalization: } \sum_{\text{hidden vars}} \phi(X', \text{hidden}, Z') = \phi(X', Z')$$

$$\textcircled{3} \text{ Conditioning: dropping any rows inconsistent with the evidence.}$$

X	Y	Z	$\phi(X, Y, Z)$
0	0	0	0.08
0	0	1	0.31
0	1	0	0.09
0	1	1	0.37
1	0	0	0.01
1	0	1	0.05
1	1	0	0.02
1	1	1	0.07

$Y = 1$

X	Z	$\phi(X, Z)$
0	0	0.09
0	1	0.37
1	0	0.02
1	1	0.07

Naive Bayes Model

Example: $Q \rightarrow O$

$$P(Q|O') = \frac{P(Q, O')}{P(O')} = \frac{P(Q')P(O'|Q')}{P(O')}$$

Generalized Naive model using "plate"



// Assumed Naive because all observations are assumed conditionally indep. of each other.

Sum-Product variable elimination

- ① eliminate factors that contain evidence variables using factor conditioning
- ② marginalize OUT the hidden factors
- ③ use factor product to consolidate any remaining factors.

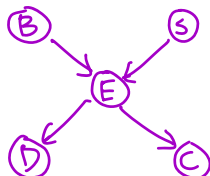
Direct Sampling

A straightforward method of 'just counting through samples'

$$P(b|c', d') = \frac{\text{count}(b', c', d')}{\text{count}(c', d')}$$

Likelihood Weighted Samples: instead of equally weighting, weigh by conditional probability given the observed PARENT!

Ex:



recall this is a topological sort.

$$P(b^1 | d^1, c^1) = \frac{0.95}{0.95 + 0.95 + 0.01 + 0.01 + 0.95} \approx 0.331$$

	B	S	E	D	C	Weight
1	0	1	1	1	$P(d^1 e^1)P(c^1 e^1)$	
0	1	1	1	1	$P(d^1 e^1)P(c^1 e^1)$	
0	0	0	1	1	$P(d^1 e^0)P(c^1 e^0)$	
0	0	0	1	1	$P(d^1 e^0)P(c^1 e^0)$	
0	0	1	1	1	$P(d^1 e^1)P(c^1 e^1)$	

Inference in Gaussian Models.

Two, jointly-distr. Gaussian Vars

$$\begin{bmatrix} a \\ b \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} A & C \\ C^T & B \end{bmatrix} \right) \quad \text{where} \quad \begin{matrix} a \sim \mathcal{N}(\mu_a, A) \\ b \sim \mathcal{N}(\mu_b, B) \end{matrix}$$

closed-form solution

$$\begin{aligned} P(a|b) &= \mathcal{N}(a | \mu_{a|b}, \Sigma_{a|b}) \\ \mu_{a|b} &= \mu_a + CB^{-1}(b - \mu_b) \\ \Sigma_{a|b} &= A - CB^{-1}C^T \end{aligned}$$

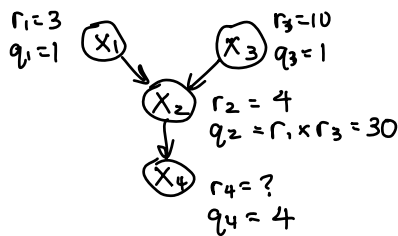
Parameter Learning $\hat{\theta} = \operatorname{argmax}_{\theta} P(D|\theta) \rightarrow$ optimize the parameters θ to fit data D
(sounds like machine learning)

① Maximum likelihood parameter learning

idea: find $l(\theta)$ and take $\frac{\partial}{\partial \theta} (l(\theta))$

② MLE for Gaussian distributions $\rightarrow$ read book

③ Parameter Learning for Bayes Nets - idea: learn the probabilities in a Bayes Net.



Nomenclature

$X_{1:n}$ - Variables (nodes in the Bayes Net)

$O_{1:n}$ - observations of instantiations of the Bayes Net

G - graph structure (largely irrelevant)

r_i - # of instantiations of x_i

q_i - # of parental permutation instantiations of x_i

π_{ij} - j^{th} instantiation of parents of x_i

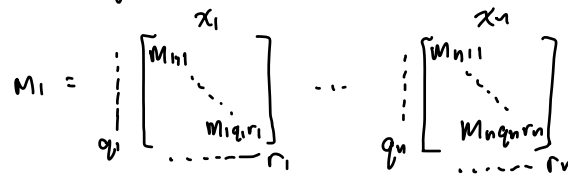
Define parameters:

$$\theta_{ijk} \triangleq P(x_i = k | \pi_{ij})$$

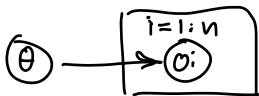
"probability x_i takes value k given j^{th} parental instantiation of x_i "

$$\theta_{ijk} = \frac{m_{ijk}}{\sum_k m_{ijk}}$$

where $m_{ijk} \triangleq \text{count} \left(\begin{smallmatrix} x_i \text{ takes value } k \text{ given } j^{\text{th}} \text{ parental} \\ \text{instantiation of } x_i \end{smallmatrix} \right)$



Bayesian Parameter Learning (an inference problem using a Naive Bayesian Model)



Beta distribution:

$$\text{Beta}(\alpha, \beta) = \frac{\Gamma(\alpha) \Gamma(\beta)}{\Gamma(\alpha + \beta)}$$

Prior: $\text{Beta}(\alpha, \beta)$ where $\alpha = \beta = 1$ if uniform prior

Observe: m_1 positive outcomes
 m_2 negative outcomes

Post: $\text{Beta}(\alpha + m_1, \beta + m_2)$

$$\text{Mode}(\text{Beta}) = \frac{\alpha - 1}{\alpha + \beta - 2}$$

$$\text{Mean}(\text{Beta}) = \frac{\alpha}{\alpha + \beta}$$

Dirichlet Distribution

$$\text{Dir}(\theta_{1:n} | \alpha_{1:n}) \triangleq \frac{\Gamma(\alpha_0)}{\prod_i \Gamma(\alpha_i)} \prod_i \theta_i^{\alpha_i - 1}$$

$$\text{where } \alpha_0 = \sum_n \alpha_{1:n}$$

$$\text{Mean}_i(\text{Dir}) = \frac{\alpha_i}{\sum_{j=1}^n \alpha_j}$$

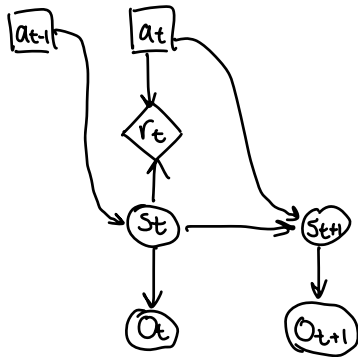
$$\text{Mode}_i(\text{Dir}) = \frac{\alpha_i - 1}{\sum_{j=1}^n \alpha_j - n}$$

Mean of Dir is a VECTOR

Beliefs

Discrete State Filter

$O(o | a, s')$ $\triangleq$ probability of observing o when action a is taken, and state s' is transitioned to.



// general structure of a POMDP problem.

$b(s)$ $\triangleq$ probability (or probability density for continuous states) assigned to state s .

for discrete states $b(s) = \vec{b} \in \mathcal{B} \in \mathbb{R}^{|S|}$

$\downarrow$ belief vector $\downarrow$ belief space $\downarrow$ $|S|$ -space
 "belief simplex"

$$\vec{b} \geq 0 \quad \mathbf{1}^T \vec{b} = 1 \rightarrow \text{sum of values in } \vec{b} = 1$$

If an agent with current belief b , takes action a , and results in observation o , what is the new belief $b'(s')$?

$$\begin{aligned}
 b'(s') &\triangleq P(s' | b, a, o) \\
 &\propto P(o | b, a, s') P(s' | b, a) && \text{Bayes' Rule} \\
 &= O(o | a, s') P(s' | b, a) && o \text{ is conditionally ind. of } b \\
 &= O(o | a, s') \sum_s P(s' | b, a, s) P(s | b, a) && \text{Law of Tot. Prob.} \\
 \star &= O(o | a, s') \sum_s T(s' | a, s) b(s) && \text{Transition is cond. ind. of belief } b, \\
 & && \& \quad b(s) \triangleq P(s | b, a)
 \end{aligned}$$

$\downarrow$ Notice this method requires accurate observation & transition models!

Kalman Filter | Adopt $\star$ to be able to handle continuous distr.

Key Assumptions : T, O, b follow a linear Gaussian distribution.

$$\begin{aligned} T(s'|s, a) &= \mathcal{N}(s' | T_s s + T_a a, \Sigma_s) \\ O(o|s') &= \mathcal{N}(o | O_s s', \Sigma_o) \\ b(s) &= \mathcal{N}(s | \mu_b, \Sigma_b) \end{aligned} \quad \begin{aligned} T_s, T_a &\triangleq \begin{matrix} \text{state} \\ \text{transition matrices} \end{matrix} \\ O_s &\triangleq \text{observation transition matrix} \end{aligned}$$

Predict Step : move the dynamics forward without considering any observations.

$$\begin{aligned} \mu_p &\leftarrow T_s \mu_b + T_a a \\ \Sigma_p &\leftarrow T_s \Sigma_b T_s^T + \Sigma_s \end{aligned}$$

Update Step : update the underlying belief distribution. (uses observations)

$$\begin{aligned} K &\leftarrow \Sigma_p O_s^T (O_s \Sigma_p O_s^T + \Sigma_o)^{-1} \quad // \text{KALMAN GAIN} \\ \mu_b &\leftarrow \mu_p + K (o - O_s \mu_p) \\ \Sigma_b &\leftarrow (I - K O_s) \Sigma_p \end{aligned}$$

Extended Kalman Filter $\rightarrow$ for systems where dynamics are nonlinear

$$\begin{aligned} T(s'|s, a) &= \mathcal{N}(s' | f_T(s, a), \Sigma_s) \\ O(o|s') &= \mathcal{N}(o | f_o(s'), \Sigma_o) \end{aligned} \quad \left. \vphantom{\begin{aligned} T(s'|s, a) &= \mathcal{N}(s' | f_T(s, a), \Sigma_s) \\ O(o|s') &= \mathcal{N}(o | f_o(s'), \Sigma_o) \end{aligned}} \right\} \begin{aligned} &\text{where } f_T \text{ \& } f_o \text{ are} \\ &\text{now nonlinear functions} \end{aligned}$$

Unscented Kalman Filters | $x \rightarrow f(x) \rightarrow x'$

$$\begin{aligned} x &\begin{cases} \rightarrow \mu \\ \rightarrow \Sigma \end{cases} & x' &\begin{cases} \rightarrow \mu' \\ \rightarrow \Sigma' \end{cases} \end{aligned}$$

$\downarrow$
some nonlinear
fun.

find the effect of transforming x through a nonlinear function $f(x)$

$$\begin{aligned} \mu &= \sum_i w_i s_i & \xrightarrow{f} & \mu' = \sum_i w_i f(s_i) \\ \Sigma &= \sum_i w_i (s_i - \mu)(s_i - \mu)^T & & \Sigma' = \sum_i w_i (f(s_i) - \mu')(f(s_i) - \mu')^T \end{aligned}$$

Where s_i are a specially selected set of "sigma points"
& w_i are the weights asoc. with the sigma points.

$$s_1 = \mu$$

$$s_{2i} = \mu + (\sqrt{(n+\lambda)\Sigma})_i \quad \text{for } i \text{ in } 1:n$$

$$s_{2i+1} = \mu - (\sqrt{(n+\lambda)\Sigma})_i \quad \text{for } i \text{ in } 1:n$$

$$w_i = \begin{cases} \frac{\lambda}{n+\lambda} & \text{for } i=1 \\ \frac{1}{2(n+\lambda)} & \text{otherwise} \end{cases}$$

$\lambda \triangleq$ spread parameter

Particle Filter → an approximation method. USEFUL for large discrete state spaces or non-Gaussian continuous state spaces

- >> begin with m sampled states s_i from underlying belief. → stochastic step
- >> propagate each s_i to s_i' using $T(s_i' | s_i, a)$.
- >> weight each s_i' by $O(o | s_i', a)$ → AKA weight each propagated state by the probability you observed what you did given said s' state.
- >> recurse on WEIGHTED s_i'

sanity check on why this works

Particle Filter with Rejection → for problems with discrete observation

idea: keep propagating

Notion of Particle Deprivation: due to the stochastic nature of particle resampling, it is easy to lose particles in relevant regions of the state space.

Soln: Particle Injection ⇐ you may inject simply a const. # of new particles.

Adaptive Injection:

$w_{\text{mean}} \triangleq$ current mean particle weight

$$w_{\text{fast}} \leftarrow w_{\text{fast}} + \alpha_{\text{fast}} (w_{\text{mean}} - w_{\text{fast}})$$

$$w_{\text{slow}} \leftarrow w_{\text{slow}} + \alpha_{\text{slow}} (w_{\text{mean}} - w_{\text{slow}})$$

where

$$0 \leq \alpha_{\text{slow}} < \alpha_{\text{fast}} \leq 1$$

→ 2 exponential moving averages to track

$$m_{\text{inject}} = \lfloor m \max(0, 1 - \nu \frac{w_{\text{fast}}}{w_{\text{slow}}}) \rfloor$$

where $\lfloor x \rfloor \triangleq$ round to the nearest integer.

$\nu \geq 1$ is an injection rate tuner.

Exact Belief State Planning

Belief-State MDP: All POMDPs can be considered MDPs that use beliefs as states

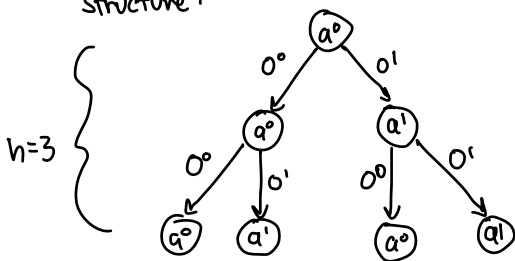
Reward function of belief-state MDP

$$R(b, a) = \sum_s R(s, a) b(s) \quad // \text{Quick Sanity Check}$$

Belief-State MDPs are always challenging to solve since the state-space is continuous BY DEFINITION!

CONDITIONAL PLANS:

Structure:



Nodes are actions

edges are OBSERVATIONS

$\pi()$ $\rightarrow$ plan associated w/ the root node

$\pi(o)$ $\rightarrow$ SUB-plan assoc. w/ the observation o

Algo

$\gg$ Start at root state s and execute $\pi()$

$\gg$ traverse to s' child state and execute $\pi(o)$

($\hookrightarrow$) repeat until decision horizon h is reached.

Suppose we have conditional plan p & we want to compute expected utility when starting from state s .

$$U^\pi(s) = R(s, \pi()) + \gamma \left[\sum_{s'} T(s'|s, \pi()) \sum_o O(o|s', \pi()) \underbrace{U^{\pi(o)}(s')}_{\text{recursive element}} \right]$$

//Reminiscent of Bellman update

This value - function update rule ASSUMES we know what state (s) we are in, what if we are working with beliefs?

$$U^\pi(b) = \sum_s U^\pi(s) b(s)$$

$$U^*(b) = \max_{\pi} \sum_s U^\pi(s) b(s) \quad // \text{optimal } h\text{-step value fn.}$$

$|A|^{|S|} \rightarrow \# \text{ of MDP policies (every state can take } |A| \text{ actions)}$

$\frac{(|O|^h - 1)}{|O| - 1} \rightarrow \# \text{ of Nodes in an } h\text{-step conditional plan}$
 (can think of nodes as states) $\frac{(|O|^h - 1)}{|O| - 1} \sim |S|$

Hence $|A|^{\frac{(|O|^h - 1)}{|O| - 1}} = \# \text{ of possible conditional plans}$

// Hugh - Jass number (intractable in h)

ALPHA VECTORS | Idea: vectorize the utility function of each state under some plan π

$$\vec{\alpha}_\pi = [U^\pi(s_1), U^\pi(s_2), \dots, U^\pi(s_m)] \in \mathbb{R}^{|S|}$$

Equipped with this: $U^\pi(b) = \sum_s U^\pi(s) b(s) = \boxed{\alpha_\pi^T b(s)}$

WAYS to represent a POMDP policy

① Conditional Plan

② Alpha Vectors tagged with actions

$\Gamma \triangleq \text{set of all alpha vectors}$

$= \{\alpha_{a_1}, \alpha_{a_2}, \dots, \alpha_{a_n}\} \rightarrow \text{where } a_i \triangleq \pi() \text{ // action at root of conditional plan.}$

How to execute policy represented by Γ ?

$\gg b' = \text{Update}(b, a, o)$

$\gg \alpha^* = \arg\max_{\alpha} \alpha^T b'$ for all α in Γ

$\gg \text{perform action associated with } \alpha^*$

③ Alpha Vectors with 1-step lookahead

one-step lookahead action from belief b using Γ with value function denoted by U^Γ

$$\pi^\Gamma(b) = \arg\max_a [R(b, a) + \gamma \sum_o P(o|b, a) U^\Gamma(\text{Update}(b, a, o))]$$

where $R(b, a) = \sum_s R(s, a) b(s)$

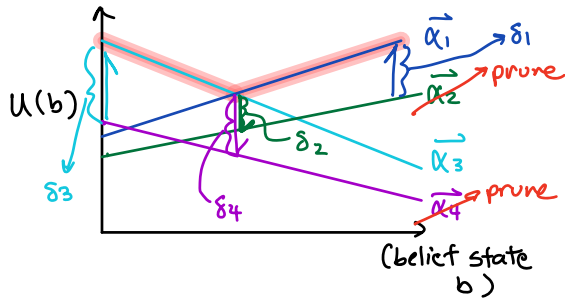
$P(o|b, a) = \sum_s P(o|s, a) b(s)$

$P(o|s, a) = \sum_{s'} T(s'|s, a) O(o|s', a)$

Pruning → Idea: Remove Alpha Vectors that would never be considered optimal regardless of belief b .

$$\underset{\delta, b}{\text{maximize}} \quad \delta \quad \text{subject to} \quad \alpha^T b \geq \alpha'^T b + \delta \quad \forall \alpha' \in \Gamma$$

→ Returns a belief b , & a gap δ



iterate over all belief states b for each alpha vector α .

notice how δ_3, δ_1 are both ≥ 0 , meaning SOMEWHERE along the belief continuum, α_3 & α_1 are optimal.

δ_2, δ_4 are both ≤ 0 NOWHERE along the belief continuum are they optimal.

→ Get that weak shit outta here (prune)

Value Iteration Idea: Cycle of extend plan depth → prune

- ① Compute all $\vec{\alpha}$ of a k -step plan
 - ② Prune all dominated $\vec{\alpha}$ of a k -step plan
 - ③ Extend to a $(k+1)$ -step plan with all non-dominated $\vec{\alpha}$ from the k -step plan as sub-plans
- Repeat

Chapter 15 Cheatsheet Exploration & Exploitation

Philosophy of exploration vs exploitation

exploration seeks to BUILD A COMPREHENSIVE MODEL

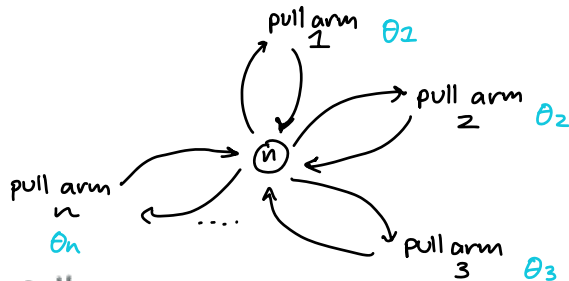
exploitation seeks to TAKE ADVANTAGE of what currently seems like the best action(s)

Bayesian Model Exploration (multi-arm lever problem)

θ_a indicates probability of success on pulling arm a .

$$\mathcal{S} = \{n\}$$

$$\mathcal{A} = \{1, 2, \dots, n\} \quad // \text{arm to pull}$$



Posterior Probability of success of arm a : p_a where $w_a \triangleq \# \text{ of observed wins}$
 $l_a \triangleq \# \text{ of observed losses}$

$$p_a = P(\text{win}_a | w_a, l_a) = \int_0^1 \theta * \text{Beta}(\theta | w_a + 1, l_a + 1) d\theta = \theta_a = \frac{w_a + 1}{w_a + l_a + 2}$$

starting with a uniform prior

Undirected Exploration ● $\rightarrow$ denotes strategy hyperparameter

ϵ -greedy $\rightarrow$ explore random arm (action) w/ prob ϵ .

(Alg 15.3) $\rightarrow$ explore $a = \arg\max_a p_a$ w/ prob $1 - \epsilon$.

$\uparrow \epsilon$: move exploration

What if you want more exploitation later?

A: Decay ϵ by α factor every time.

Explore-then-commit $\rightarrow$ explore a random arm for first K steps

(Alg 15.4) $\rightarrow$ take greedy action AFTER K steps.

Directed Exploration → unlike undirected, uses info gathered from previous steps.

● → denotes strategy hyperparameter

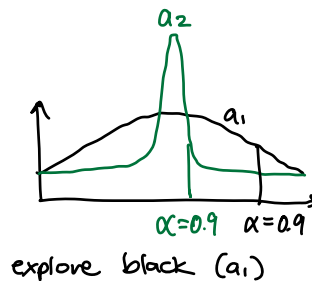
softmax exploration (Alg 15.5)

pull arm a with probability $\propto \exp(\lambda p_a)$

- when $\lambda = 0$, p_a is irrelevant and effectively sampling at uniform across all a
(purely explorative)
- when $\lambda \rightarrow \infty$, only choosing $a = \arg\max_a p_a$
(purely exploitative)

quantile exploration

- choose arm w/ the largest $(\alpha > x)$ quantile.
- $\alpha \uparrow \rightarrow$ more explorative



Upper confidence Bound

- choose next action a s.t. $a = \arg\max_a (p_a + c \sqrt{\frac{\log N}{N(a)}})$

where $N(a) \triangleq$ # times action a has been tried

$$N \triangleq \sum_a N(a)$$

$$c \geq 0 \Rightarrow \uparrow c \rightarrow \text{explorative}$$

Thompson Sampling | PARAMETERLESS

- utilize the posterior sample
- sample one of each action from the posterior distr.
 - choose action with greatest reward to keep exploring

Optimal Exploration (Ex 15.4)

$Q^*(w_1, l_1, \dots, w_n, l_n, a) \triangleq$ expected payoff of pulling arm a , then acting optimally.

$$U^*(w_1, l_1, \dots, w_n, l_n) = \max_a Q^* \quad // \text{optimal utility fn.}$$

$$\pi^*(w_1, l_1, \dots, w_n, l_n) = \operatorname{argmax}_a Q^* \quad // \text{optimal policy}$$

$$Q^*(w_1, l_1, \dots, w_n, l_n, a) = \frac{w_a + 1}{w_a + l_a + 2} (1 + U^*(\dots, w_{a+1}, l_a, \dots)) \rightarrow \text{winning term} \\ + \left(1 - \frac{w_a + 1}{w_a + l_a + 2}\right) (0 + U^*(\dots, w_a, l_{a+1}, \dots)) \rightarrow \text{losing term.}$$

Assuming finite horizon h ,

- start with all terminal belief states where $\sum_a (w_a + l_a) = h$
- back up 1-level in the tree to $\sum_a (w_a + l_a) = h-1$
 - apply highlighted term
- recurse.

Time complexity $O(h^{2n}) \rightarrow$ # of possible belief states.

(pretty computationally taxing)

MODEL-BASED methods RL

Philosophy of Model-Based Methods. $\rightarrow$ try to find underlying dynamics $T(s'|s,a)$ and the rewards $R(s,a)$ of the model.

MAX LIKELIHOOD METHOD $\rightarrow$ idea is simply a counting exercise.

(Alg 16.1 ; Ex 16.1)

$$T(s'|a,s) = \frac{N(s',a,s)}{N(s,a)} \quad \text{where } N(a,s) = \sum_{s'} N(s',a,s)$$

$$R(s,a) = \frac{p(s,a)}{N(s,a)} \quad \text{where } p(s,a) \text{ is the sum of all rewards obtained when taking state-action pair } (a,s)$$

IF $N(s,a) = 0 \Rightarrow T(s'|a,s) = R(s,a) = 0$
 $\rightarrow$ indicates fully unexplored state-action pair

Update Schemas $\rightarrow$ updating the value function after the model changes.

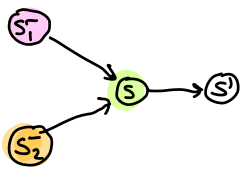
① Full Update $\rightarrow$ completely re-solve for your value fun. (computationally taxing)
NOT REC.

② Randomized Updates $\rightarrow$ hyperparameter

Update the value function at the state s you just visited

Update " " at an additional m random states.

③ Prioritized Updates



$\gg$ while num_updates < n $\rightarrow$ hyperparameter

$\gg$ update $U(s)$

$\gg$ let $u = U^*(s)$ // the OLD utility of s

$\gg$ for s^- in $\{s_1^-, s_2^-\}$

priority = $T(s|s^-,a) * |U(s) - u|$ * the key step

s^- priority = priority

$\gg$ update $U(\text{Max}_{\text{priority}}(s^-))$

$\gg$ num_updates ++

$\swarrow$
updates previous state that has highest transition probability.

Exploration that doesn't focus on actions from states beside current

R-Max exploration

$$R(s,a) = \begin{cases} r_{\max} & \text{if } N(s,a) < m \\ \rho(s,a)/N(s,a) & \text{otherwise} \end{cases}$$

where $r_{\max}$ is the maximum achievable reward.

$$T(s'|a,s) = \begin{cases} (s'=s) & \text{if } N(s,a) < m \\ N(s,a,s')/N(a,s) & \text{otherwise} \end{cases}$$

stay in the same state if underexplored.

value function of underexplored states ($N(s,a) < m$) $\rightarrow \frac{r_{\max}}{1-\gamma}$
// infinite series problem

BAYESIAN METHODS

we have model parameters $\vec{\theta}$ that can provide information on $T(s'|a,s)$ & $R(s,a)$ (we focus on $T(s'|a,s)$)

Ex: let $\vec{\theta}_{(s,a)} = [T(s_1|s,a), T(s_2|s,a), \dots, T(s_n|s,a)]$

$$\text{Dir}(\vec{\theta}_{(s,a)} | \vec{N}_{(s,a)}) \star$$

where $\vec{N}_{(s,a)} = [N(s_1,a,s), N(s_2,a,s), \dots, N(s_n,a,s)]$

// $\vec{N}(s,a) = [1, 1, \dots, 1]$ for a uniform prior

$$b(\theta) = \prod_s \prod_a \text{Dir}(\vec{\theta}_{(s,a)} | \vec{N}_{(s,a)})$$

$\rightarrow$ distribution over θ ?

Posterior Sampling (Ex 16.4)

- >> draw sample θ
- >> update belief b
- >> draw new sample θ_{update} from b
- >> create MDP problem from θ_{update}
- >> solve for a new value function of created MDP

MODEL - FREE METHODS

instead of trying to build a model with $T(s'|a,s)$ & $R(s,a)$,
try to solve for the action-value function $Q(s,a)$ directly.

Incremental Estimation of the Mean (expectation of a single variable)

$$\hat{x}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad \text{expectation of a single variable in } m \text{ samples (analogous to } Q)$$

(Derived in book)

$$\hat{x}_m = \hat{x}_{m-1} + \frac{1}{m} (x^{(m)} - \hat{x}_{m-1})$$

(generalized)

$$\hat{x}_m = \hat{x}_{m-1} + \alpha(m) (x^{(m)} - \hat{x}_{m-1}) \quad \star \text{ General incremental update rule}$$

when $\alpha(m) = \frac{1}{m} \Rightarrow$ we have incremental mean estimation

when $\alpha(m) = \text{const} \Rightarrow$ exponentially decaying past values

$$\sum_{m=1}^{\infty} \alpha(m) = \infty$$

ensures α is
BIG enough

$$\sum_{m=1}^{\infty} \alpha^2(m) < \infty$$

ensures α is
SMALL enough

Q-learning $\rightarrow$ incremental estimation of the action-value function $Q(s,a)$

$$Q(s,a) = R(s,a) + \gamma \sum_{s'} T(s'|s,a) U(s')$$

$$= R(s,a) + \gamma \sum_{s'} T(s'|s,a) \max_{a'} Q(s',a')$$

$\Updownarrow$ rewrite as expectation over samples of reward r & next state s' ★ crucial sanity check.

$$= \mathbb{E}_{r,s'} [r + \gamma \max_{a'} Q(s',a')]$$

$\rightarrow$ apply general incremental update rule

$$Q(s,a) \leftarrow \underbrace{Q(s,a)}_{\hat{x}^{(m-1)}} + \underbrace{\alpha (r + \gamma \max_{a'} Q(s',a') - Q(s,a))}_{x^{(m)}} \quad \hat{x}^{(m-1)}$$

Important to have intuition on how modulating α will impact the rate of convergence.

$\alpha \uparrow \rightarrow$ decays too slowly, very noisy convergence

$\alpha \downarrow \rightarrow$ decays too quickly, new sample $x^{(m)}$ has very little impact on $\hat{x}^{(m)}$ $\rightarrow$ slow rise time

SARSA] (s, a, r, s', a') $\rightarrow$ newly introduced from Q-learning

Update Rule
simply drops
"max"
a'

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

// must work alongside a suitable exploration strategy
to determine the next action a' to try.

If expl. strat is greedy $a' = \arg\max_{a'} (Q(s', a'))$

& SARSA becomes Q-learning!

OFFLINE POMDP METHODS

QMDP: A fully-observable Approximation

Upper bound method

constructs an $|S|$ alpha vector for each action $\alpha_a(s)$

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_{s'} T(s'|s,a) \max_{a'} \alpha_{a'}^{(k)}(s') \quad O(|A|^2 |S|^2)$$

// initialize $\alpha_a = \vec{0}$

QMDP provides an UPPER-BOUND on the optimal value function because it assumes FULL-OBSERVABILITY

Another way to get an upper bound on a value function is to set it yourself.

BABS: best-action - best state

$$\bar{U}(b) = \max_{s,a} \frac{R(s,a)}{1-\gamma} \quad \text{// take optimal reward for an infinitely discounted horiz.}$$

FAST-INFORMED BOUND

Upper bound method

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_{s'} \max_{a'} \sum_{o|a,s'} O(o|a,s') T(s'|s,a) \alpha_{a'}^{(k)}(s') \quad O(|S|^2 |A|^2 |O|)$$

// similar to QMDP but it takes into acct. the observation model

→ Typically tighter, but NEVER LOOSER upper bound than QMDP.

FAST LOWER BOUNDS

BAWS: best-action - worst state.

Lower Bound

$$\Gamma_{\text{baws}} = \underline{U}(b) = \frac{\max_a \min_s R(s,a)}{1-\gamma} \quad \begin{matrix} \rightarrow \text{take best action at worst} \\ \text{state forever} \\ \text{(tighter than WAWS)} \end{matrix}$$

$O(|S| |A|)$

Blind Lower Bound

Lower Bound

$$\alpha_a^{(k+1)}(s) = R(s,a) + \gamma \sum_{s'} T(s'|s,a) \alpha_a^{(k)}(s')$$

// very similar to QMDP, but you're missing the $\max_{a'} \alpha_{a'}^{(k)}(s')$ term
in other words, you STICK TO ONE ACTION ALWAYS

PBVI : Point - based value iteration

Slow LB (but tighter)

$$P = (\alpha_1, \dots, \alpha_m)$$